

eda-cimbtr

March 3, 2025

```
[1]: import pandas as pd
df = pd.read_csv("/kaggle/input/equity-post-HCT-survival-predictions/train.csv")
```

1 basic data exploration

```
[2]: # Shape and first few rows
print(df.shape)
print(df.head())

# Check for missing values
missing_values = df.isnull().sum().sort_values(ascending=False)
print("Missing Values:\n", missing_values[missing_values > 0])

# Data types of columns
print(df.info())

# Summary statistics
print(df.describe())
```

(28800, 60)

	ID	dri_score	psych_disturb	cyto_score	diabetes	\
0	0	N/A - non-malignant indication	No	NaN	No	
1	1	Intermediate	No	Intermediate	No	
2	2	N/A - non-malignant indication	No	NaN	No	
3	3	High	No	Intermediate	No	
4	4	High	No	NaN	No	

	hla_match_c_high	hla_high_res_8	tbi_status	arrhythmia	\
0	NaN	NaN	No TBI	No	
1	2.0	8.0	TBI +- Other, >cGy	No	
2	2.0	8.0	No TBI	No	
3	2.0	8.0	No TBI	No	
4	2.0	8.0	No TBI	No	

	hla_low_res_6	...	tce_div_match	donor_related	\
0	6.0	...	NaN	Unrelated	
1	6.0	...	Permissive mismatched	Related	

2	6.0	...	Permissive mismatched	Related
3	6.0	...	Permissive mismatched	Unrelated
4	6.0	...	Permissive mismatched	Related

	melphalan_dose	hla_low_res_8	cardiac	hla_match_drb1_high	\
0	N/A, Mel not given	8.0	No	2.0	
1	N/A, Mel not given	8.0	No	2.0	
2	N/A, Mel not given	8.0	No	2.0	
3	N/A, Mel not given	8.0	No	2.0	
4	MEL	8.0	No	2.0	

	pulm_moderate	hla_low_res_10	efs	efs_time
0	No	10.0	0.0	42.356
1	Yes	10.0	1.0	4.672
2	No	10.0	0.0	19.793
3	No	10.0	0.0	102.349
4	No	10.0	0.0	16.223

[5 rows x 60 columns]

Missing Values:

tce_match	18996
mrd_hct	16597
cyto_score_detail	11923
tce_div_match	11396
tce_imm_match	11133
cyto_score	8068
hla_high_res_10	7163
hla_high_res_8	5829
hla_high_res_6	5284
hla_match_dqb1_high	5199
hla_low_res_10	5064
conditioning_intensity	4789
hla_match_c_high	4620
hla_match_a_high	4301
hla_nmdp_6	4197
hla_match_dqb1_low	4194
hla_match_b_high	4088
hla_low_res_8	3653
hla_match_drb1_high	3352
hla_low_res_6	3270
hla_match_c_low	2800
hla_match_drb1_low	2643
hla_match_b_low	2565
cardiac	2542
peptic_ulcer	2419
hla_match_a_low	2390
arrhythmia	2202
rheum_issue	2183

```

rituximab                2148
pulm_severe              2135
diabetes                 2119
psych_disturb            2062
pulm_moderate            2047
hepatic_mild             1917
renal_issue              1915
hepatic_severe           1871
donor_age                1808
obesity                  1760
prior_tumor              1678
melphalan_dose           1405
karnofsky_score          870
cmv_status               634
ethnicity                587
comorbidity_score        477
sex_match                261
vent_hist                259
in_vivo_tcd              225
gvhd_proph               225
donor_related            158
dri_score                154

```

```
dtype: int64
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 28800 entries, 0 to 28799
```

```
Data columns (total 60 columns):
```

#	Column	Non-Null Count	Dtype
0	ID	28800 non-null	int64
1	dri_score	28646 non-null	object
2	psych_disturb	26738 non-null	object
3	cyto_score	20732 non-null	object
4	diabetes	26681 non-null	object
5	hla_match_c_high	24180 non-null	float64
6	hla_high_res_8	22971 non-null	float64
7	tbi_status	28800 non-null	object
8	arrhythmia	26598 non-null	object
9	hla_low_res_6	25530 non-null	float64
10	graft_type	28800 non-null	object
11	vent_hist	28541 non-null	object
12	renal_issue	26885 non-null	object
13	pulm_severe	26665 non-null	object
14	prim_disease_hct	28800 non-null	object
15	hla_high_res_6	23516 non-null	float64
16	cmv_status	28166 non-null	object
17	hla_high_res_10	21637 non-null	float64
18	hla_match_dqb1_high	23601 non-null	float64
19	tce_imm_match	17667 non-null	object

20	hla_nmdp_6	24603	non-null	float64
21	hla_match_c_low	26000	non-null	float64
22	rituximab	26652	non-null	object
23	hla_match_drb1_low	26157	non-null	float64
24	hla_match_dqb1_low	24606	non-null	float64
25	prod_type	28800	non-null	object
26	cyto_score_detail	16877	non-null	object
27	conditioning_intensity	24011	non-null	object
28	ethnicity	28213	non-null	object
29	year_hct	28800	non-null	int64
30	obesity	27040	non-null	object
31	mrd_hct	12203	non-null	object
32	in_vivo_tcd	28575	non-null	object
33	tce_match	9804	non-null	object
34	hla_match_a_high	24499	non-null	float64
35	hepatic_severe	26929	non-null	object
36	donor_age	26992	non-null	float64
37	prior_tumor	27122	non-null	object
38	hla_match_b_low	26235	non-null	float64
39	peptic_ulcer	26381	non-null	object
40	age_at_hct	28800	non-null	float64
41	hla_match_a_low	26410	non-null	float64
42	gvhd_proph	28575	non-null	object
43	rheum_issue	26617	non-null	object
44	sex_match	28539	non-null	object
45	hla_match_b_high	24712	non-null	float64
46	race_group	28800	non-null	object
47	comorbidity_score	28323	non-null	float64
48	karnofsky_score	27930	non-null	float64
49	hepatic_mild	26883	non-null	object
50	tce_div_match	17404	non-null	object
51	donor_related	28642	non-null	object
52	melphalan_dose	27395	non-null	object
53	hla_low_res_8	25147	non-null	float64
54	cardiac	26258	non-null	object
55	hla_match_drb1_high	25448	non-null	float64
56	pulm_moderate	26753	non-null	object
57	hla_low_res_10	23736	non-null	float64
58	efs	28800	non-null	float64
59	efs_time	28800	non-null	float64

dtypes: float64(23), int64(2), object(35)

memory usage: 13.2+ MB

None

/usr/local/lib/python3.10/dist-packages/pandas/io/formats/format.py:1458:

RuntimeWarning: invalid value encountered in greater

has_large_values = (abs_vals > 1e6).any()

/usr/local/lib/python3.10/dist-packages/pandas/io/formats/format.py:1459:

RuntimeWarning: invalid value encountered in less

```
has_small_values = ((abs_vals < 10 ** (-self.digits)) & (abs_vals > 0)).any()
/usr/local/lib/python3.10/dist-packages/pandas/io/formats/format.py:1459:
```

RuntimeWarning: invalid value encountered in greater

```
has_small_values = ((abs_vals < 10 ** (-self.digits)) & (abs_vals > 0)).any()
```

	ID	hla_match_c_high	hla_high_res_8	hla_low_res_6 \
count	28800.000000	24180.000000	22971.000000	25530.000000
mean	14399.500000	1.764516	6.876801	5.143322
std	8313.988213	0.431941	1.564313	1.207757
min	0.000000	0.000000	2.000000	2.000000
25%	7199.750000	2.000000	6.000000	4.000000
50%	14399.500000	2.000000	8.000000	6.000000
75%	21599.250000	2.000000	8.000000	6.000000
max	28799.000000	2.000000	8.000000	6.000000

	hla_high_res_6	hla_high_res_10	hla_match_dqb1_high	hla_nmdp_6 \
count	23516.000000	21637.000000	23601.000000	24603.000000
mean	5.109202	8.617230	1.736876	5.160346
std	1.214162	1.905125	0.447687	1.203240
min	0.000000	3.000000	0.000000	2.000000
25%	4.000000	7.000000	1.000000	4.000000
50%	6.000000	10.000000	2.000000	6.000000
75%	6.000000	10.000000	2.000000	6.000000
max	6.000000	10.000000	2.000000	6.000000

	hla_match_c_low	hla_match_drb1_low	...	age_at_hct \
count	26000.000000	26157.000000	...	28800.000000
mean	1.757808	1.715296	...	38.663162
std	0.435453	0.451282	...	21.147581
min	0.000000	1.000000	...	0.044000
25%	2.000000	1.000000	...	19.539000
50%	2.000000	2.000000	...	41.006000
75%	2.000000	2.000000	...	55.965250
max	2.000000	2.000000	...	73.726000

	hla_match_a_low	hla_match_b_high	comorbidity_score	karnofsky_score \
count	26410.000000	24712.00000	28323.000000	27930.00000
mean	1.709087	1.69962	1.702327	83.83208
std	0.458259	0.46518	1.994443	11.02884
min	0.000000	0.00000	0.000000	40.00000
25%	1.000000	1.00000	0.000000	70.00000
50%	2.000000	2.00000	1.000000	90.00000
75%	2.000000	2.00000	2.000000	90.00000
max	2.000000	2.00000	10.000000	100.00000

	hla_low_res_8	hla_match_drb1_high	hla_low_res_10	efs \
count	25147.000000	25448.000000	23736.000000	28800.000000

mean	6.903448	1.707128	8.664687	0.539306
std	1.565017	0.461179	1.882746	0.498461
min	2.000000	0.000000	4.000000	0.000000
25%	6.000000	1.000000	7.000000	0.000000
50%	8.000000	2.000000	10.000000	1.000000
75%	8.000000	2.000000	10.000000	1.000000
max	8.000000	2.000000	10.000000	1.000000

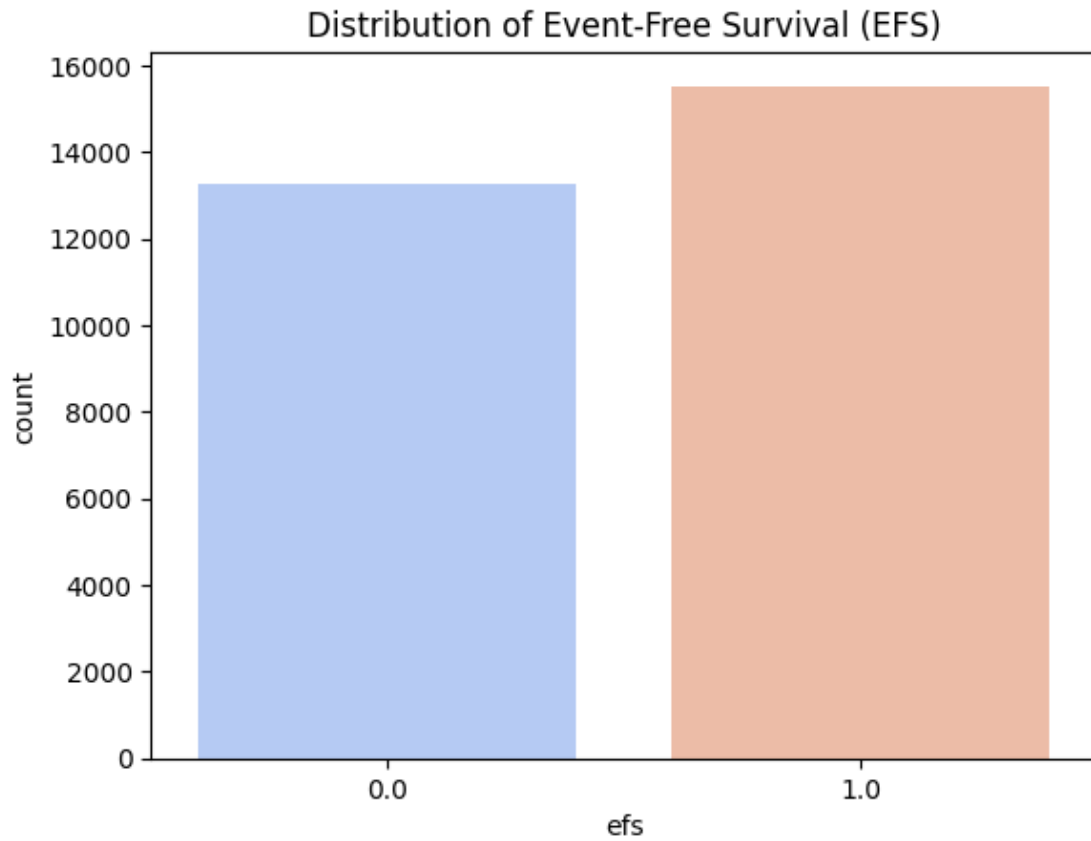
	efs_time
count	28800.000000
mean	23.237678
std	24.799748
min	0.333000
25%	5.619750
50%	9.796500
75%	35.100000
max	156.819000

[8 rows x 25 columns]

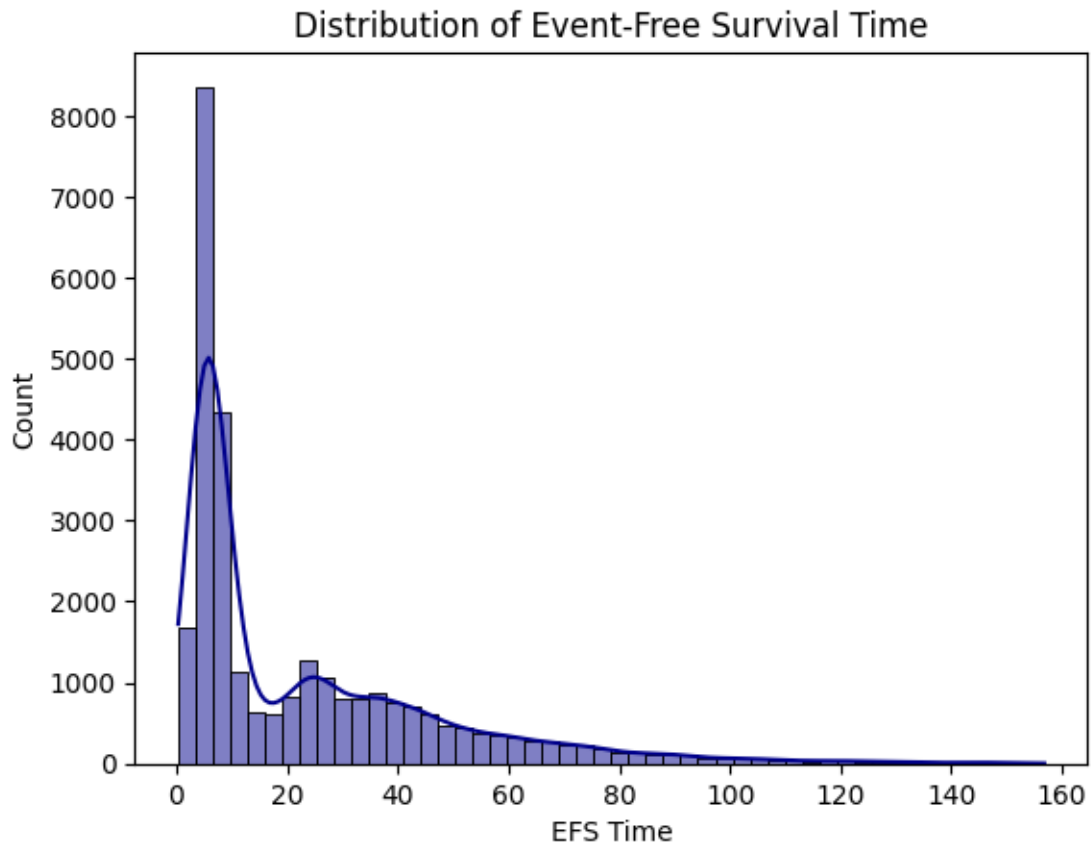
Target Variable Analysis

```
[5]: # Event-Free Survival (Binary Target)
import seaborn as sns
import matplotlib.pyplot as plt
sns.countplot(x="efs", data=df, palette="coolwarm")
plt.title("Distribution of Event-Free Survival (EFS)")
plt.show()

# Distribution of EFS Time
sns.histplot(df["efs_time"], bins=50, kde=True, color="darkblue")
plt.title("Distribution of Event-Free Survival Time")
plt.xlabel("EFS Time")
plt.show()
```



```
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1119: FutureWarning:  
use_inf_as_na option is deprecated and will be removed in a future version.  
Convert inf values to NaN before operating instead.  
  with pd.option_context('mode.use_inf_as_na', True):
```



2 for efs:

Observation: The dataset has a fairly balanced distribution between patients who had event-free survival and those who did not.

3 for efs time:

Observation: The majority of patients have a short event-free survival time, with a right-skewed distribution indicating a small

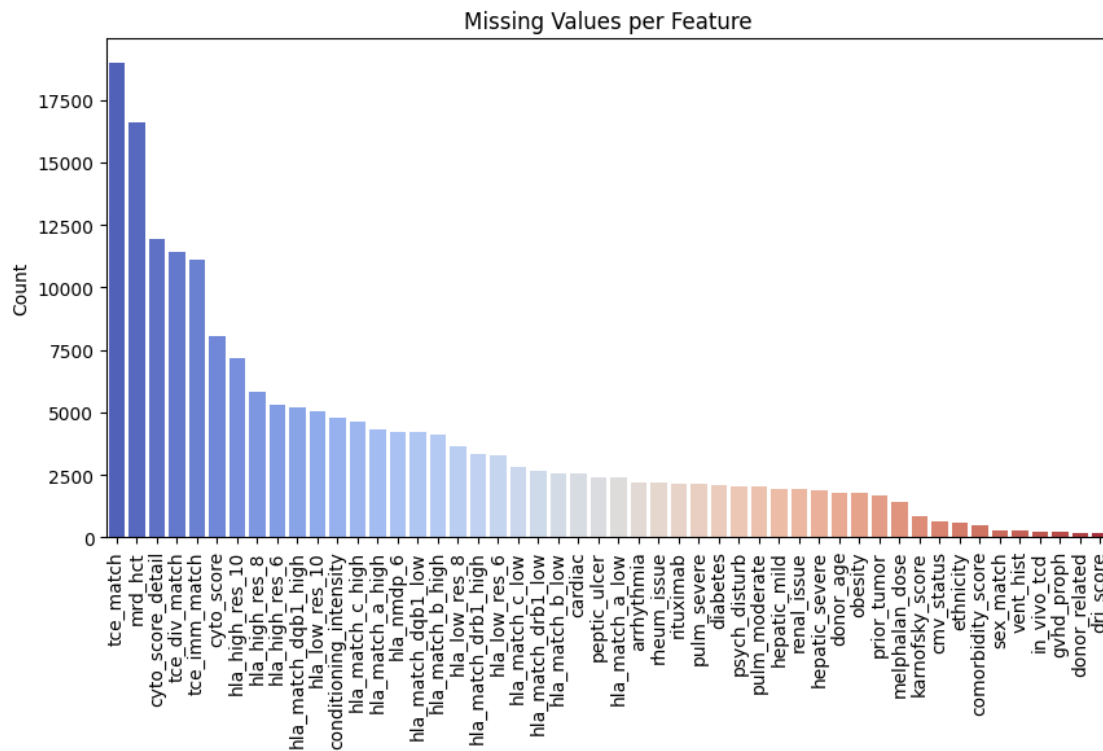
subset of patients who experience long-term event-free survival.

Missing Values Analysis

```
[6]: missing_values = df.isnull().sum().sort_values(ascending=False)
missing_values = missing_values[missing_values > 0]

plt.figure(figsize=(10,5))
sns.barplot(x=missing_values.index, y=missing_values.values, palette='coolwarm')
plt.xticks(rotation=90)
```

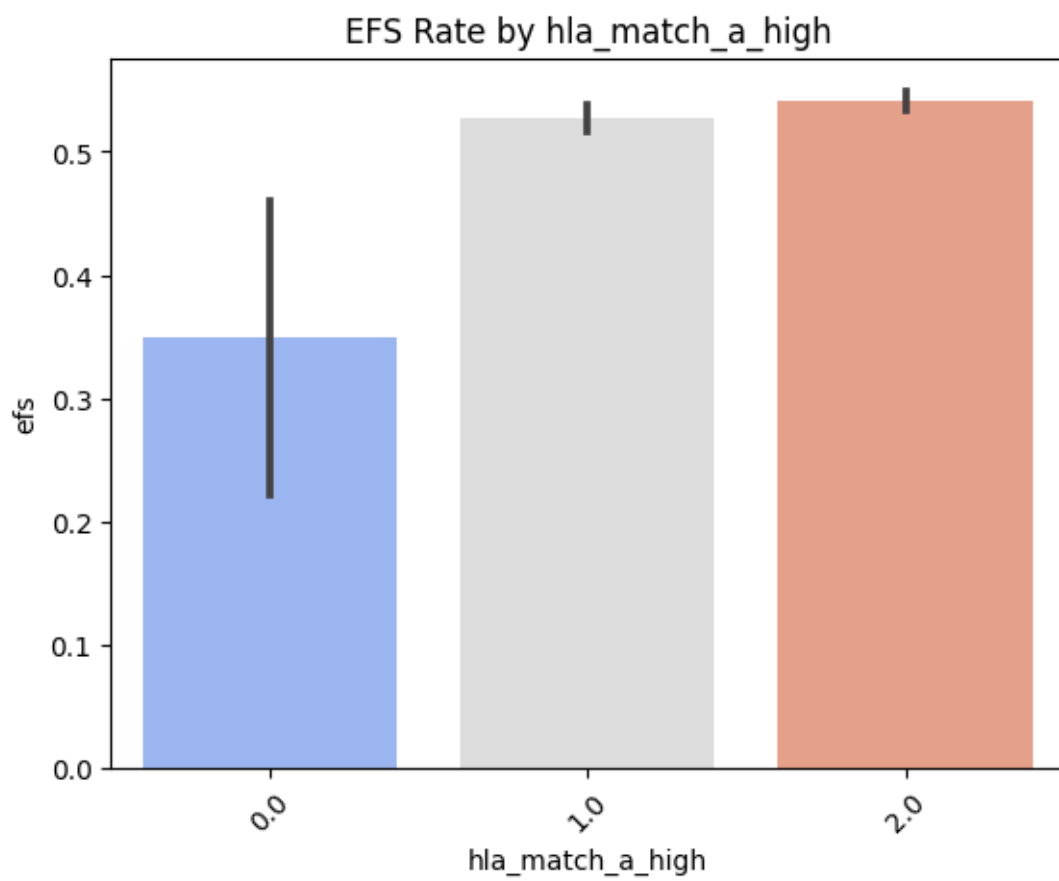
```
plt.title("Missing Values per Feature")
plt.ylabel("Count")
plt.show()
```

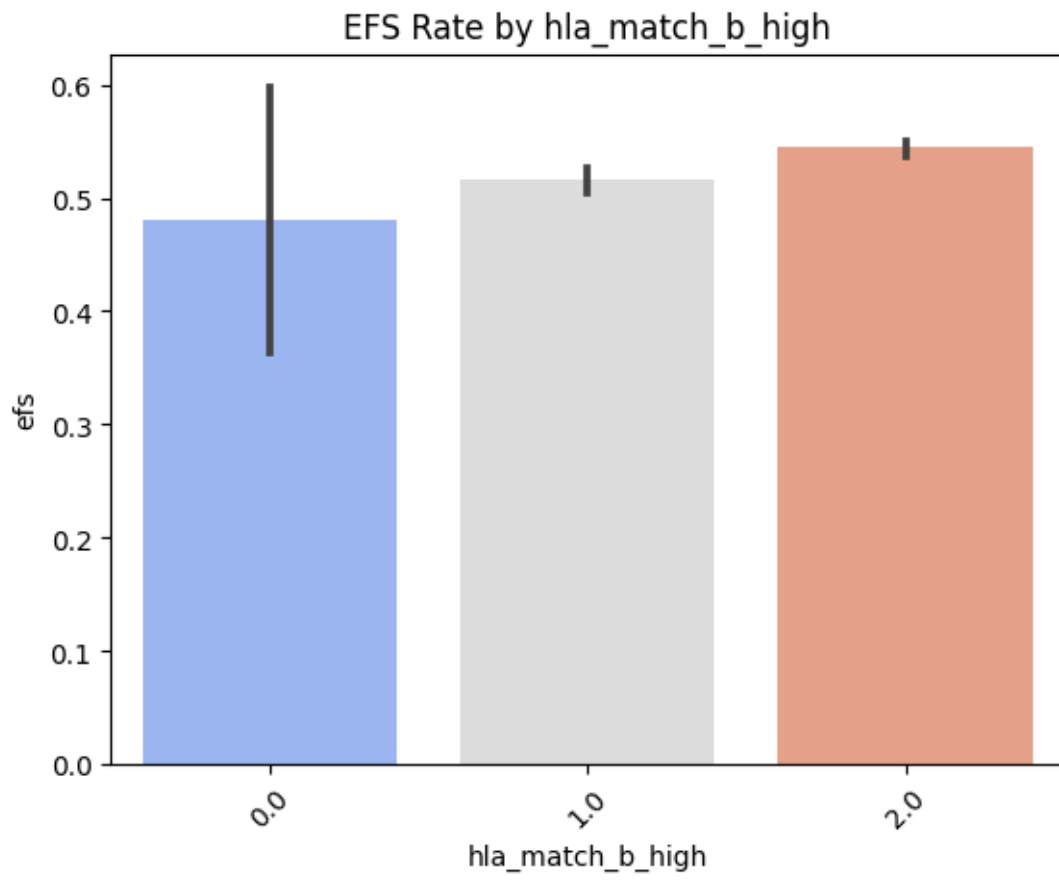


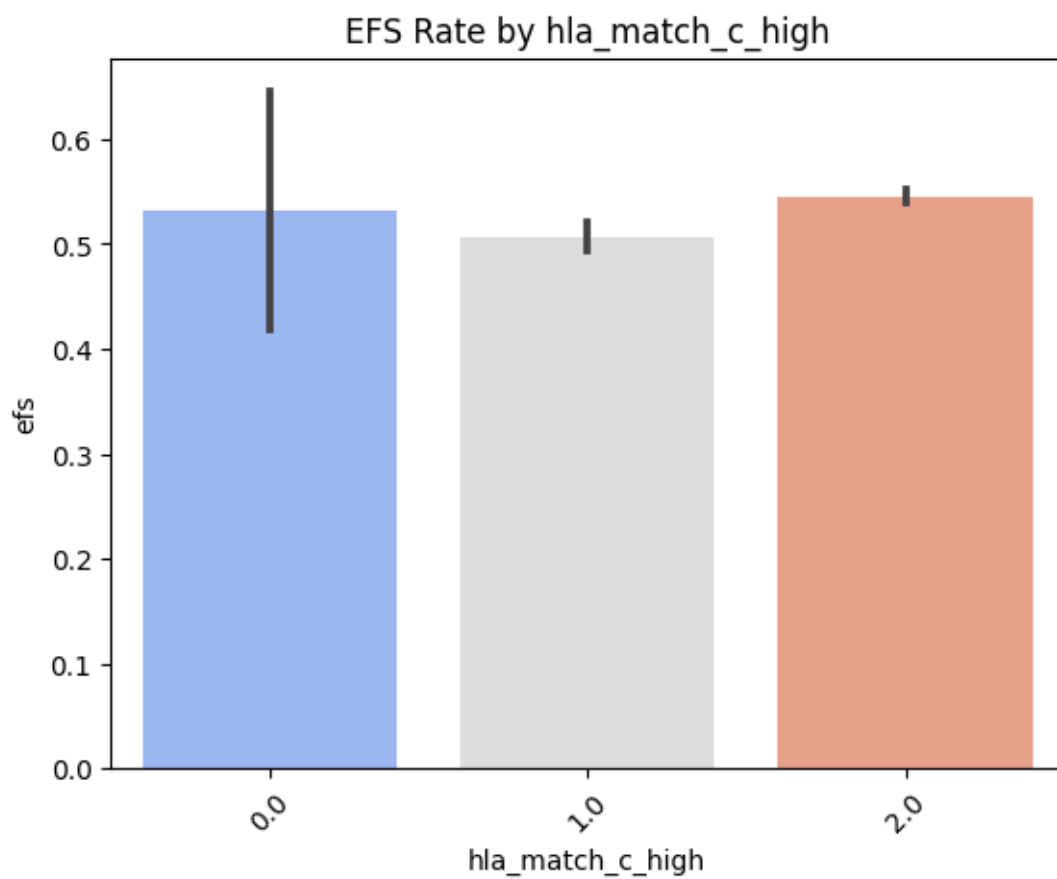
Observation: Some features have missing values, which may require imputation or removal.

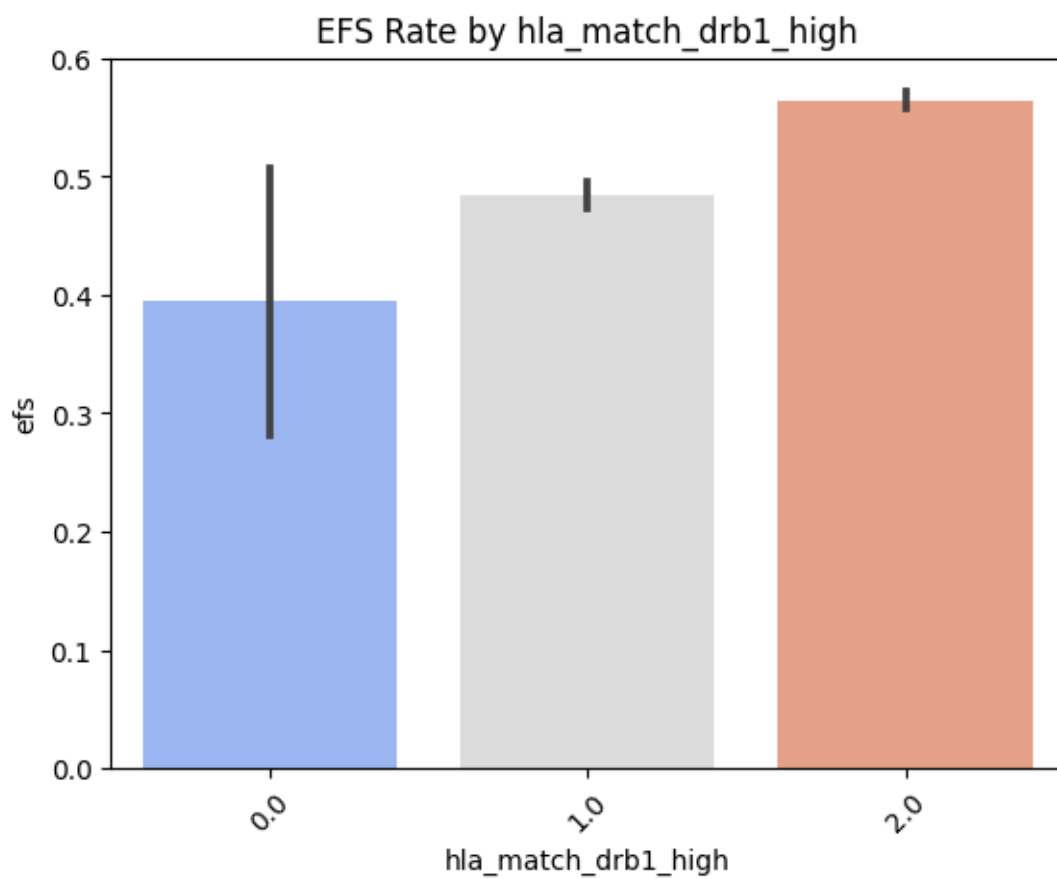
```
[8]: hla_cols = ["hla_match_a_high", "hla_match_b_high", "hla_match_c_high",
               ↪ "hla_match_drb1_high", "hla_match_dqb1_high"]

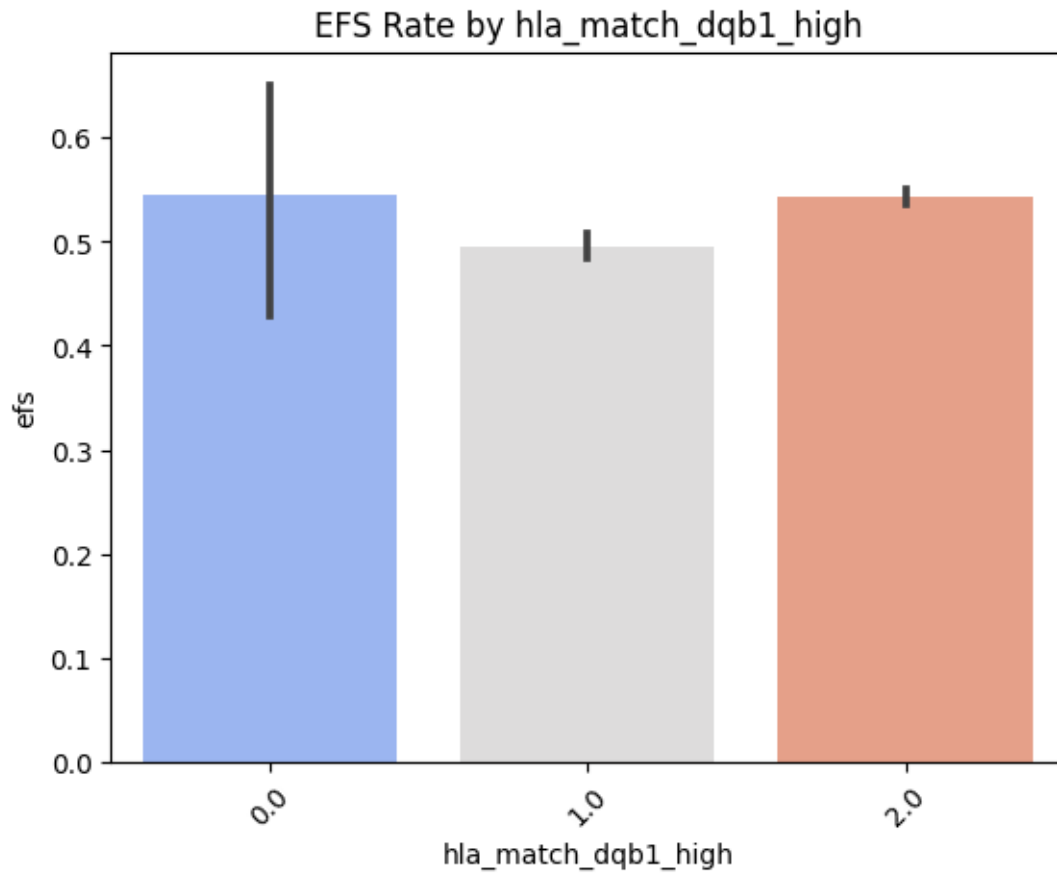
for col in hla_cols:
    sns.barplot(x=col, y="efs", data=df, palette="coolwarm")
    plt.xticks(rotation=45)
    plt.title(f"EFS Rate by {col}")
    plt.show()
```





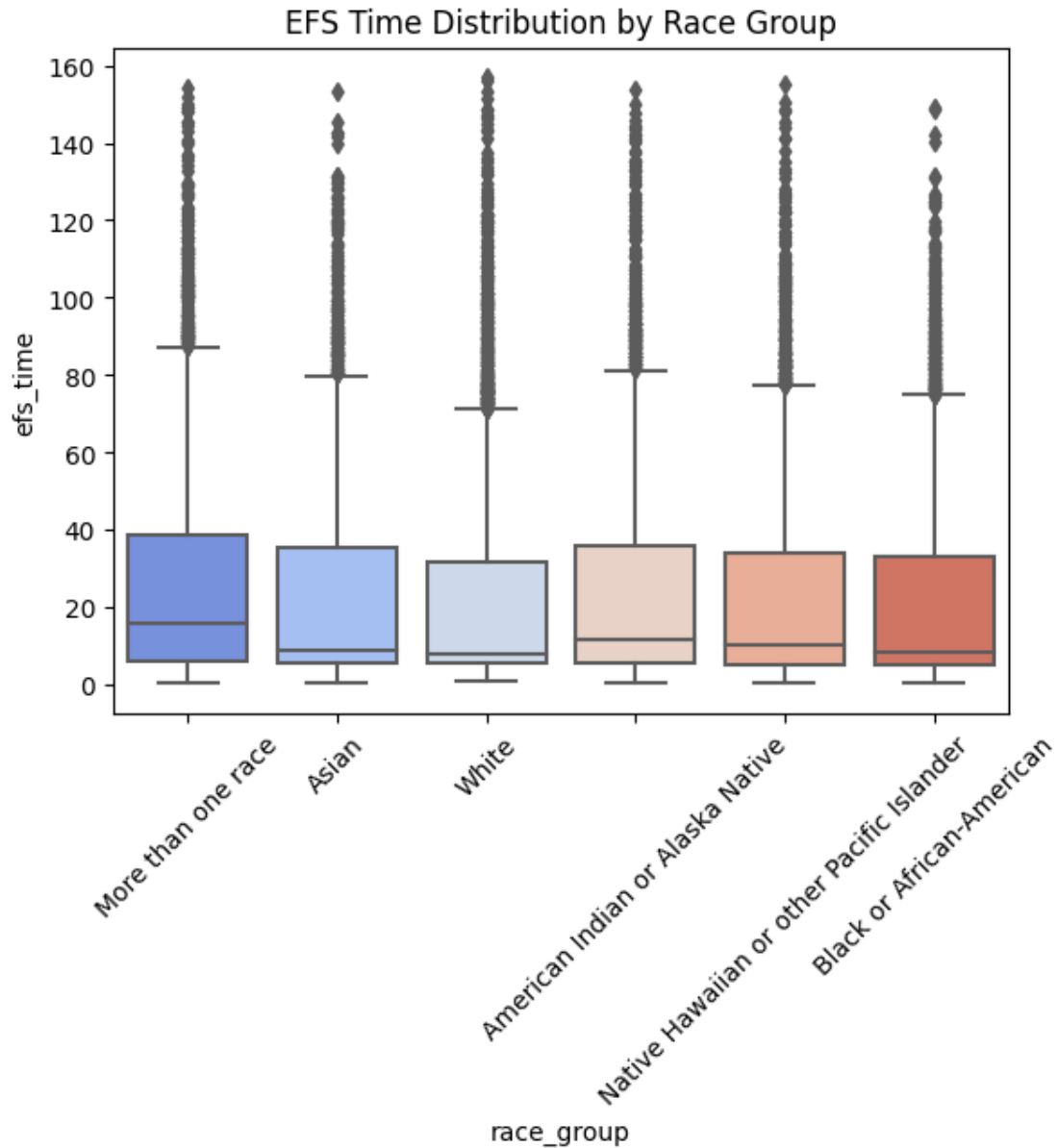






Observation: so in some cases there are some difference efs but in most the cases differences are not very significant

```
[9]: sns.boxplot(x="race_group", y="efs_time", data=df, palette="coolwarm")  
plt.title("EFS Time Distribution by Race Group")  
plt.xticks(rotation=45)  
plt.show()
```



Observation:

The median EFS time appears relatively consistent across all race groups.

There is a high presence of outliers in all categories, indicating variability in survival times.

The interquartile range (IQR) suggests that most patients experience EFS times within a similar range, regardless of race.

The presence of numerous outliers above the upper whisker suggests that some individuals have significantly longer EFS times than the majority.

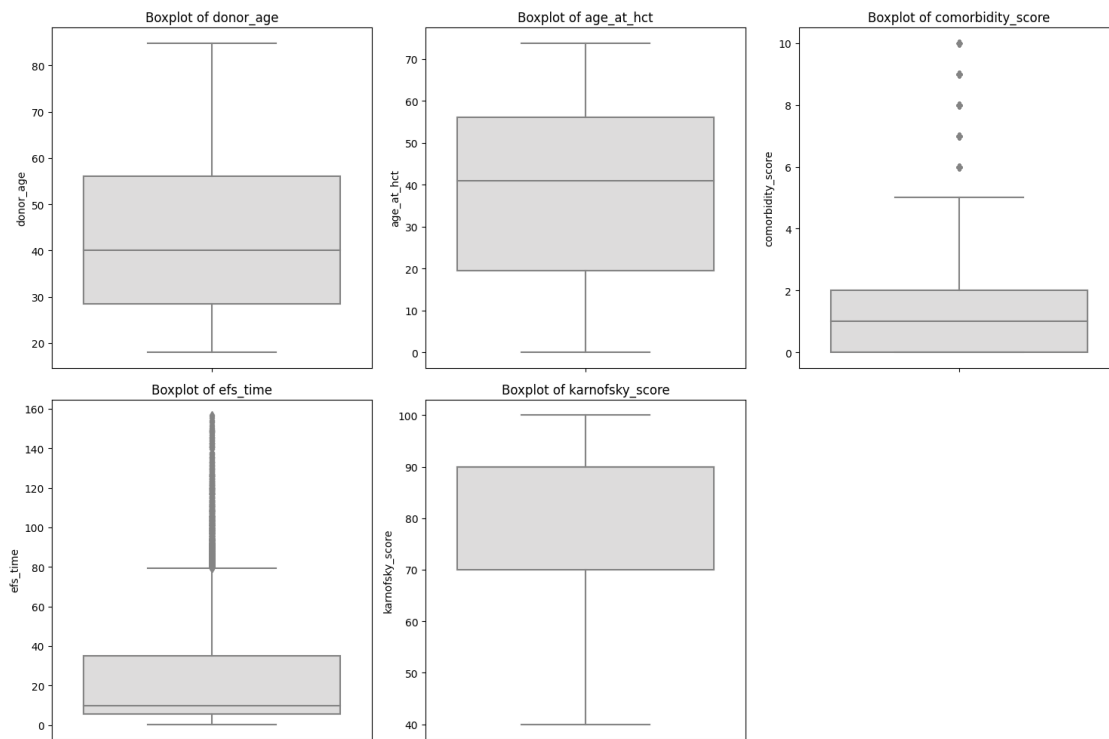
```
[12]: import matplotlib.pyplot as plt
import seaborn as sns

num_cols = ["donor_age", "age_at_hct", "comorbidity_score", "efs_time",
            ↪ "karnofsky_score"]
fig, axes = plt.subplots(2, 3, figsize=(15, 10))
axes = axes.flatten()

for i, col in enumerate(num_cols):
    sns.boxplot(y=col, data=df, ax=axes[i], palette="coolwarm")
    axes[i].set_title(f"Boxplot of {col}")

# Hide the unused subplot
for j in range(len(num_cols), len(axes)): # Hide any extra subplot
    fig.delaxes(axes[j])

plt.tight_layout()
plt.show()
```



Observation:

Donor Age & Age at HCT: The distributions appear fairly normal, with the median age around 40–50 years. The age range varies significantly, with a few outliers.

Comorbidity Score: Most patients have low comorbidity scores, but a few outliers indicate signifi-

cantly higher scores. EFS Time: Similar to the previous visualization, EFS time has a high number of outliers, suggesting large variability in survival duration.

Karnofsky Score: Most patients have a high Karnofsky score (above 70), indicating good functional status.

4 Principal Component Analysis (PCA)

```
[16]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

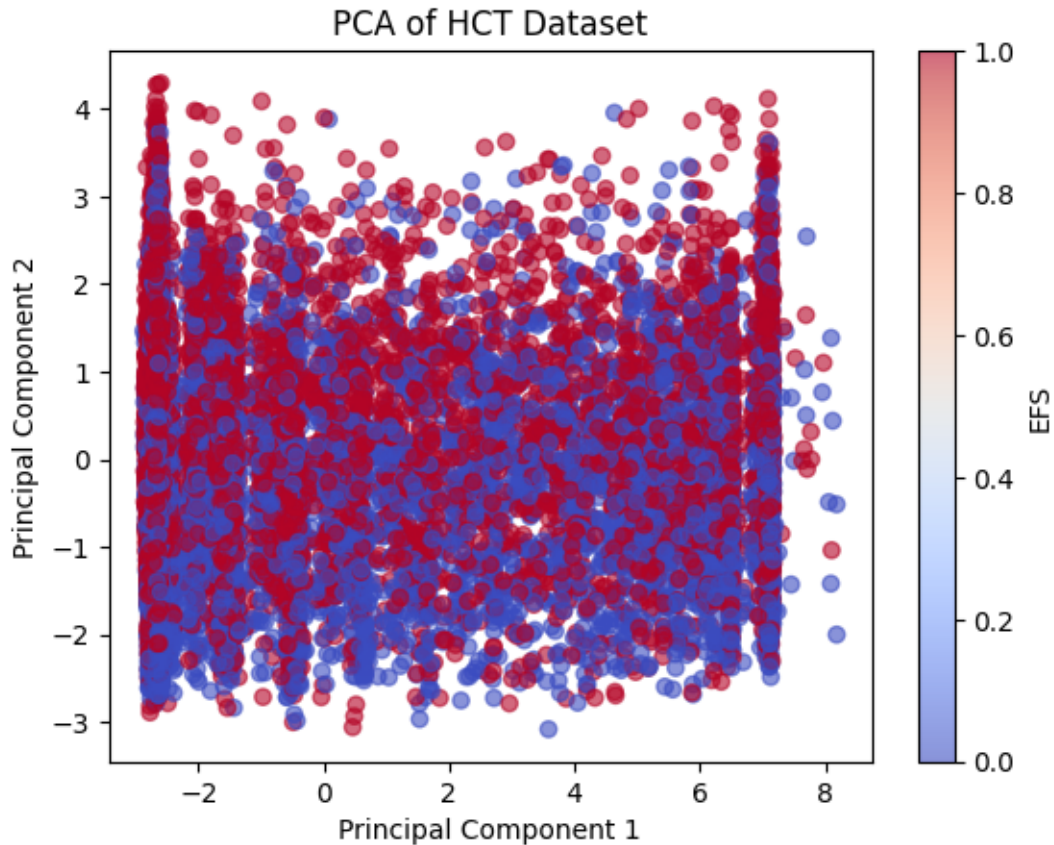
# Drop rows with missing values in features and target variables together
df_cleaned = df.dropna(subset=["efs", "efs_time"]) # Ensure target variables
↳ have no NaNs
features = df_cleaned.select_dtypes(include=[np.number]).drop(columns=["efs",
↳ "efs_time"])

# Drop rows with NaN in features to keep dimensions consistent
df_cleaned = df_cleaned.loc[features.dropna().index] # Ensure same index
features = features.dropna()

# Standardize the features
scaler = StandardScaler()
features_scaled = scaler.fit_transform(features)

# Apply PCA
pca = PCA(n_components=2)
pca_features = pca.fit_transform(features_scaled)

# Ensure color labels match the transformed data
plt.scatter(pca_features[:, 0], pca_features[:, 1], c=df_cleaned["efs"].values,
↳ cmap="coolwarm", alpha=0.6)
plt.title("PCA of HCT Dataset")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.colorbar(label="EFS")
plt.show()
```



Observation:

Above figure shows very high correlation between `efs` and `efs_time`.

5 Clustering Analysis

```
[18]: from sklearn.cluster import KMeans
import seaborn as sns
import matplotlib.pyplot as plt

# Drop rows with missing values in features and target variables together
df_cleaned = df.dropna(subset=["efs", "efs_time"]) # Ensure target variables
↳ have no NaNs
features = df_cleaned.select_dtypes(include=[np.number]).drop(columns=["efs",
↳ "efs_time"])

# Drop rows with NaN in features to keep dimensions consistent
df_cleaned = df_cleaned.loc[features.dropna().index] # Ensure same index
features = features.dropna()
```

```

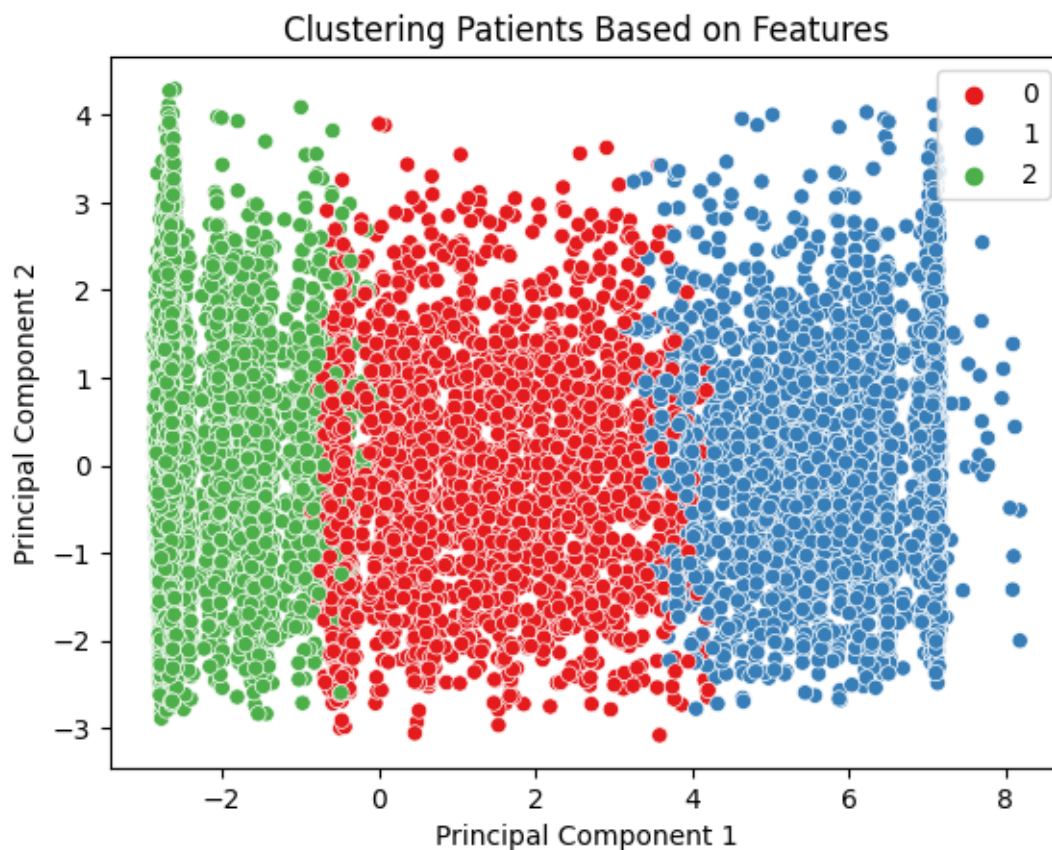
# Standardize the features
scaler = StandardScaler()
features_scaled = scaler.fit_transform(features)

# Apply K-Means Clustering
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10) # Add `n_init` to
↳avoid warning
df_cleaned["cluster"] = kmeans.fit_predict(features_scaled) # Assign clusters
↳to the cleaned dataframe

# PCA for visualization
pca = PCA(n_components=2)
pca_features = pca.fit_transform(features_scaled)

# Plot PCA clusters
sns.scatterplot(x=pca_features[:, 0], y=pca_features[:, 1],
↳hue=df_cleaned["cluster"], palette="Set1")
plt.title("Clustering Patients Based on Features")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.legend()
plt.show()

```



observation : so it shows using efs time we can predict efs values were not completely overlaped